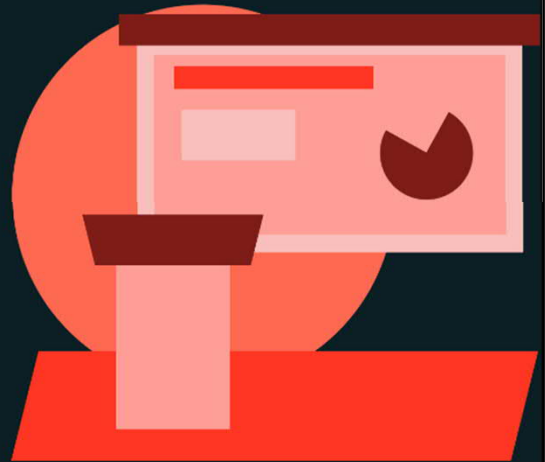




Introduction to Apache Spark™

LECTURE

Introduction to Apache Spark DataFrames and SQL



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Introduction to Apache Spark DataFrames and SQL: Learn how to work with structured data using Spark's DataFrame API and SQL for efficient querying, transformation, and analysis.

Lesson Learning Objectives

- Understand the DataFrame, Spark's structured data abstraction, featuring schema enforcement, optimized execution, and SQL compatibility.
- Learn to transform, filter, and aggregate data using both DataFrame operations and Spark SQL expressions.
- Learn how to read from and write to various formats (CSV, JSON, Parquet), handling schema inference and data type management.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://apache.org/).

By the end of this lecture, you will be able to understand the DataFrame, Spark's structured data abstraction, featuring schema enforcement, optimized execution, and SQL compatibility; transform, filter, and aggregate data using both DataFrame operations and Spark SQL expressions; and read from and write to various formats (CSV, JSON, Parquet) while handling schema inference and data type management.

What is a DataFrame

Structured data processing in Apache Spark

- DataFrames are distributed collections of records, all with the same pre-defined structure (schema).
- Built on Spark's core concepts but with structure, optimization, and familiar SQL-like operations for data manipulation.
- DataFrames track their schema and provide native support for many of the common SQL functions and relational operators (**JOINS** etc).
- DataFrames are evaluated as DAGs, using "lazy evaluation" (evaluating DataFrames only when required) and providing lineage and fault tolerance.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

DataFrames are structured collections of records with a pre-defined schema for efficient processing. They are built on Spark's core concepts with added support for SQL-like operations and optimization. DataFrames keep track of their schema and provide native support for SQL operations like joins. They are evaluated as DAGs with lazy evaluation for optimized processing and fault tolerance (lazy evaluation will be discussed later when introducing Spark transformations and actions).

Creating DataFrames

DataFrames can be created....

- JSON, CSV, Parquet, ORC, Text or Binary Files
- Delta Lake or other Table Storage format directories
- A table or view in a catalog (like Unity Catalog)
- An external database
- An existing RDD

```
...  
path = "examples/src/main/resources/people.csv"  
df = spark.read.csv(path)  
...
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

DataFrames can be created from various sources, including JSON, CSV, Parquet, ORC, text, or binary files. They also support Delta Lake or other table storage format directories. DataFrames can be created from tables or views in Unity Catalog, external databases, or existing RDDs. For example, a CSV file can be read using `spark.read.csv()`.

DataFrame API Optimizations

- **Adaptive Query Execution** – Dynamic plan adjustments (cost-based optimization) during runtime based on actual data characteristics and execution patterns.
- **In-Memory Columnar Storage** – In-memory columnar format for all DataFrames enabling efficient analytical query performance and reduced memory footprint.
- **Built-in Statistics** – Automatic statistics collection when saving to optimized formats (Parquet, Delta) enables smarter query planning and execution.
- **Catalyst Optimizer and Photon** – discussed on the next slide...



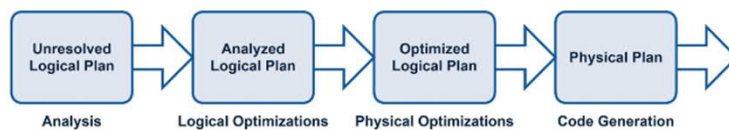
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Tungsten is Spark's columnar in-memory execution engine, providing a consistent columnar representation for DataFrames regardless of the data source. While DataFrames always use Tungsten's columnar format in memory, statistics are only collected when saving to optimized file formats. Tungsten delivers several performance benefits, including off-heap memory management, cache-aware computation, code generation for faster execution, and whole-stage code generation. It is important to distinguish between the in-memory representation, which is always columnar via Tungsten, and the on-disk storage formats, which may be row-based (e.g., CSV) or columnar (e.g., Parquet). Tungsten's columnar format is a key enabler of many of Spark's query optimizations.

DataFrame (or Query) Planning

Understanding how Spark optimizes and evaluates DataFrames

- When a DataFrame is evaluated, the Driver creates an optimized execution plan through a series of transformations – converting the logical plan into an efficient physical execution plan that minimizes resource usage and execution time



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Spark creates an optimized execution plan for DataFrames (or queries) through a series of transformations. The logical plan is analyzed and converted into a physical execution plan for efficient resource usage. This process includes Analysis, Logical Optimizations, Physical Optimizations, and Code Generation. An example diagram illustrates the progression from the Unresolved Logical Plan to the Physical Plan.

Catalyst Optimiser and Photon

Query optimization, code generation and efficient memory management

- **Catalyst Optimizer:**

- Query optimization engine that converts DataFrame operations into an optimized execution plan
- Applies rule-based and cost-based optimizations to improve query performance

- **Photon Engine:**

- Databricks-native vectorized query engine that accelerates query execution
- Processes data in batches rather than row-by-row for better performance
- Runs by default on Databricks SQL warehouses and serverless compute
- Can be enabled on All-Purpose and Job Clusters in Databricks

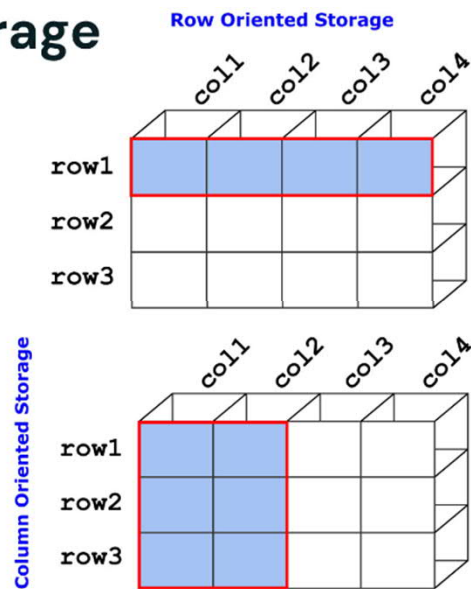


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

The Catalyst Optimizer converts DataFrame operations into optimized execution plans using both rule-based and cost-based optimizations. The Photon Engine, a Databricks-native vectorized query engine, accelerates query execution by processing data in batches rather than row-by-row. Photon is enabled by default in SQL warehouses and serverless compute, with optional support for all-purpose and job clusters.

More on Columnar Storage

- Organizes data by column rather than row, enabling efficient scanning and analysis
- Particularly efficient for analytical workloads
- Implemented in DataFrame internal storage and in physical file encoding formats such as **Parquet** and ORC



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Columnar storage organizes data by columns rather than rows, enabling efficient scanning and analysis, especially for analytical workloads with large datasets. It is implemented in DataFrame internal storage and in file formats such as Parquet and ORC, with visual comparisons often illustrating the difference between row-oriented and column-oriented storage.

DataFrameReader and DataFrameWriter

DataFrame API interfaces for data I/O

DataFrameReader

(`spark.read`)

- Supports multiple formats
- Provides schema inference

```
...
df = spark.read.format("format").option(...).load()
# or using predefined format readers
df = spark.read.csv("abfss://c1@a1...net/csv-dir")
df = spark.read.parquet("s3://mybucket/parquet-dir")
...
```

```
...
df.write.format("format").mode("mode").save()
# or using predefined format readers
df.write.csv("abfss://c1@a1...net/csv-dir")
df.write.parquet("s3://mybucket/parquet-dir")
...
```

DataFrameWriter (`dataframe.write`)

- Flexible output formats and partitioning
- Supports various save modes (overwrite, append)



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The DataFrameReader (`spark . read`) supports multiple formats and schema inference, while the DataFrameWriter (`dataframe . write`) allows flexible output formats and partitioning. It supports save modes such as overwrite and append, with code snippets demonstrating reading and writing DataFrames to various sources like CSV and Parquet.

DataFrame Schemas

Structure, types, and data guarantees

- Every DataFrame has a defined schema – structure and data types of all columns
- Can be inferred from data or explicitly specified (more efficient):

```
# Defining a schema
from pyspark.sql.types import *
schema = StructType([
    StructField("name", StringType()),
    StructField("age", IntegerType())
])
```

- Self-describing formats like Parquet include schema information
-  Use the `printSchema()` function to print out the DataFrame schema



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Every DataFrame has a defined schema specifying the structure and data types of its columns. The schema can be inferred from the data or explicitly defined for greater efficiency. Self-describing formats like Parquet automatically include schema information. An example code snippet demonstrates defining a schema using PySpark's StructType.

DDL Schemas

An alternative to StructType definitions

- Instead of defining schemas using StructType objects, you can use DDL (Data Definition Language) strings for more readable schema definitions
- You can do this...

```
# Defining a DDL schema
ddl_schema = "name STRING NOT NULL, age INT, city STRING"

# Using the schema
df = spark.read.csv("abfss://c1@a1...net/csv-dir", schema=ddl_schema)
df.printSchema()
```



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

An alternative to using StructType definitions for defining DataFrame schemas is using DDL (Data Definition Language) strings, which provide a more readable format. This approach is useful for specifying schemas when reading data files such as CSV or Parquet. An example code snippet demonstrates defining a schema with DDL strings and applying it to a DataFrame.

DataFrame Data Types

Primitive Data Types

<code>pyspark.sql.types.DataType</code>	Spark SQL Type	Python base equivalent
<code>ByteType</code>	TINYINT	<code>int</code>
<code>ShortType</code>	SMALLINT	<code>int</code>
<code>IntegerType</code>	INT	<code>int</code>
<code>LongType</code>	BIGINT	<code>long</code>
<code>FloatType</code>	FLOAT	<code>float</code>
<code>DoubleType</code>	DOUBLE	<code>float</code>
<code>BooleanType</code>	BOOLEAN	<code>bool</code>
<code>StringType</code>	STRING	<code>string</code>
<code>BinaryType</code>	BINARY	<code>bytearray</code>
<code>TimestampType</code>	TIMESTAMP	<code>datetime.datetime</code>
<code>DateType</code>	DATE	<code>datetime.date</code>



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

In terms of DataFrame data types, many of the standard data types used in programming languages and SQL are represented as data types within Spark SQL.

In the Python API (`pyspark.sql.types.DataType`), these include primitive data types such as Boolean, Integer, Long, Float, etc., along with their Spark SQL equivalents, which are used in DDL schemas and displayed when exploring SQL IDs in tools like the Databricks Catalog Explorer. Their native Python equivalents are also provided.

DataFrame Data Types

Complex Data Types

DataType	SQL Type	Python base equivalent	Notes
ArrayType	ARRAY	list (or tuple)	Ordered elements of the same type
MapType	MAP	dict	Keys and values can have different types
StructType	STRUCT	tuple, dict, namedtuple	Named fields with specified types

For more information see: https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/data_types.html



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

There is also support for complex types, such as arrays, maps, and structs. Arrays are ordered elements of the same type. Maps are keys and values that can have different types. Structs are named fields with specified types—essentially like a table with predefined columns. A reference link is available to view the various primitive and complex data types supported in the DataFrame API in Spark SQL.

Transformations and Actions

DataFrame Programming Operations

DataFrames are immutable – once created, their data cannot be modified, instead...:

- **Transformations** create new DataFrames from existing ones (can be categorized as *Coarse Grain Transformations*)
- **Actions** (like showing or saving output) trigger actual computation and produce final results
 - Multiple transformations can be called, the Job is only created when an Action is requested – a process called **Lazy Evaluation**



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Transformations create new DataFrames from existing ones and are lazy operations that define a logical plan without triggering execution. Actions initiate the actual computation, returning results to the driver or writing to external storage. Spark's lazy evaluation optimizes performance by delaying computation until an action is invoked, allowing multiple transformations to be applied before execution to create optimized execution plans.

Common DataFrame API Methods

Transformations		Actions	
<code>select()</code>	Choose specific columns from DataFrame	<code>count()</code>	Return number of rows in a DataFrame
<code>filter()</code>	Filter rows based on condition, alias <code>where()</code>	<code>show()</code>	Display DataFrame content
<code>withColumn()</code>	Add or modify a column in a DataFrame	<code>take(n)</code>	Return first n rows from a DataFrame
<code>groupBy()</code>	Group rows in a DataFrame by specified columns	<code>first()</code>	Return first row from a DataFrame
<code>agg()</code>	Aggregate rows in a DataFrame after grouping	<code>write()</code>	Save DataFrame to storage

For more information see: <https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html>



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Transformations such as `select()`, `filter()`, `withColumn()`, `groupBy()`, and `agg()` define how data is transformed but do not trigger computation. Actions such as `count()`, `show()`, `take()`, `first()`, and `write()` trigger computation and produce results.

Transformations create a new DataFrame, whereas actions return results or write data to external sources. Understanding this distinction is key to grasping Spark's lazy evaluation model.

SparkSQL

SQL Interface for Spark DataFrames

DataFrame Registration:

- Temporary views:
`createOrReplaceTempView()`
- Global temp views:
`createGlobalTempView()`
- Enables SQL queries on DataFrames

SQL Query Execution:

- Use `spark.sql()` for SQL statements
- Seamless integration with DataFrame API
- Full SQL support
- Complex types and expressions
- Built-in functions library
- Seamless metastore integration



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

DataFrame registration can be done using `createOrReplaceTempView()` for temporary views and `createGlobalTempView()` for global views. SQL query execution allows using `spark.sql()` to run SQL queries over DataFrames. Seamless integration with the DataFrame API enables combining SQL queries with DataFrame operations. This supports complex expressions, built-in functions, and metadata integration through the metastore.

Primer on SQL Metastores

Metadata Management for Spark SQL

- A metastore is responsible for...
 - defining table schemas and locations
 - defining table partitions
 - Storing properties and information for objects including tables, views, functions
- **Unity Catalog** is the centralized metadata service provided for Databricks accounts and assigned to Databricks Workspaces that enables fine-grained security and governance across all data assets



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

A metastore is responsible for defining table schemas, locations, and partitions. It stores metadata and properties for objects such as tables, views, and functions.

Unity Catalog is a centralized metadata service provided by Databricks for security and governance, enabling consistent access control and data lineage across Databricks workspaces.



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).